

A layer by hand, then in one line

Paper first. Then code. Then check the two agree digit for digit. That order is the method for the entire rest of this course, and this is the lab where it starts — so do it in that order even though you could get the answer faster the other way.

Scaffolded lab

18 minutes

Paper required

Question 1 is not optional

Your layer 1 · on paper 2 · in NumPy 3 · assert it 4 · batch it 5 · break it 6 · second layer
7 · the collapse Submission

Your layer

Open the provided `lab05.ipynb`. The first cell already holds these exact numbers — run it without changing them, so that your answers can be checked against everyone else's.

cell 1 – type it exactly

PYTHON

```
import numpy as np

X = np.array([[2.0, -1.0]])           # one example, two features
W = np.array([[1.0,  0.5, -2.0],      # two features in, three units out
               [-1.0, 2.0,  0.0]])
b = np.array([0.5, -0.5, 1.0])       # one bias per unit
```

Before anything else, say the three shapes out loud: `X` is (1, 2), `W` is (2, 3), `b` is (3,). If the multiplication rule from part 2 of the lecture is fresh, you already know the output will be (1, 3).

On paper, before touching the keyboard



Keyboard down for five minutes

Yes, one line of NumPy would give you the answer now. The entire value of this task is that you do it the slow way once, so that in session 13 — where the same computation has thirteen numbers in it and a wrong answer is invisible — you know what the machine is doing rather than trusting it.

Write out all three dot products **in full**, every term shown:

on paper, like this

BY HAND

```
unit 0:  2.0 × 1.0   + (-1.0) × (-1.0) + 0.5 = ?
unit 1:  2.0 × ___   + (-1.0) × ___     + ___ = ?
unit 2:  2.0 × ___   + (-1.0) × ___     + ___ = ?
```

Three numbers. Photograph the paper and put the image into your notebook as the answer to task 1.



Which numbers pair with which

Unit 0 uses *column* 0 of `W` — that is 1.0 and -1.0, read downward, not across. Reading a row instead of a column is the single most common error here, and it produces three plausible-looking wrong numbers. If you are unsure, write `W` out as a grid on your paper and draw the column you are using.

Now in NumPy

PYTHON

```
print(X @ W + b)
print((X @ W + b).shape)
```

If it does not match your paper, **find the discrepancy before reading on**. Do not correct the paper to match the code. The disagreement is the most useful thing this lab can give you, and it is nearly always the row-versus-column error above.

`[[3.5, -1.5, -3.0]]`, shape `(1, 3)`.

Unit 0: $2.0 \times 1.0 + (-1.0) \times (-1.0) + 0.5 = 2.0 + 1.0 + 0.5 = 3.5$. Unit 1: $2.0 \times 0.5 + (-1.0) \times 2.0 + (-0.5) = 1.0 - 2.0 - 0.5 = -1.5$. Unit 2: $2.0 \times (-2.0) + (-1.0) \times 0.0 + 1.0 = -4.0 + 0 + 1.0 = -3.0$.

TASK 3 · 1 MINUTE

Assert it

3

fill in your three hand-computed numbers

PYTHON

```
assert np.allclose(X @ W + b, [[?, ?, ?]])
print("paper and code agree")
```

An assertion that passes prints nothing and an assertion that fails stops the cell. This is how you will verify every piece of unit 2 — not by looking at output and thinking it seems about right, but by stating what you expect and letting the machine object.

TASK 4 · 3 MINUTES

Batch it

4

Add three more example rows of your own choosing, so `X` has shape `(4, 2)`.

- a Print the output shape and confirm it is what the rule `(n,k) @ (k,m) → (n,m)` predicted.
- b Confirm that row 0 of the output is *unchanged* from task 2. Say in one sentence why it must be.
- c Note that `b` is still shape `(3,)` and nothing broke. Which rule from Tuesday made that work?

TASK 5 · 2 MINUTES

Break it, and read the error

5

Change `W` to shape `(3, 2)` instead — same numbers, transposed — and run it.

Copy the **full** error message into a markdown cell, then explain in your own words which two numbers NumPy compared and why they failed to match. You will see this message more than any other this semester, and being able to read it in two seconds instead of two minutes is worth the two minutes now.

TASK 6 · 3 MINUTES

Add the second layer

6

Create `W2` of shape `(3, 1)` and `b2` of shape `(1,)`, with numbers of your choosing. Compute the full 2–3–1 forward pass on your batch of four examples.

- a Report the output shape. Four examples in, how many numbers out?
- b Count your total weights and biases. You should get **9 and 4** — the numbers from part 5 of the lecture, and the same network that appears on slide 21 of the orientation deck.

TASK 7 · 2 MINUTES · THE POINT OF THE WHOLE LAB

Show the collapse

7

Verify with `np.allclose` that your two-layer network gives exactly the same answers as a *single* layer whose weights are `W @ W2` and whose bias is `b @ W2 + b2`.

PYTHON

```
two_layer = (X @ W + b) @ W2 + b2
one_layer = X @ (W @ W2) + (b @ W2 + b2)

print(np.allclose(two_layer, one_layer))    # True
```

Then write **two sentences**: what this implies about depth, and what would have to change to fix it.



This is the most important result in unit 1

Two layers just did the work of one. A hundred stacked layers would also do the work of one, and so a network built purely from matrix multiplies gains nothing at all from depth. Everything you build in unit 2 depends on the missing ingredient that fixes this — and you have now proved the problem exists rather than been told about it.

Submission

Restart Kernel and Run All Cells so the notebook is clean top to bottom, then push to GitHub Classroom before session 9 on Tuesday.



Task 1 — the photographed paper — is not optional

Hand computation is assessed on the midterm, where there is no laptop. This lab is the practice, and the photograph is how I know you did it in the right order.